CORE JAVA SERIES — DAY 7

# Constructors, the this Keyword & Object Internals

Default and parameterized constructors, constructor overloading and chaining, the this keyword, how much memory an object actually takes, and why Java is call-by-value even for objects.

## Default Values of Instance Variables

Variables declared inside a class (fields like `name`, `age`, `rollNumber`, `college`) are called **instance variables** (they live in the heap, as part of an object). Functions written inside a class are called **instance methods**. If you create an object and never assign its fields, printing them shows Java's **default values** — not an error.

| Data Type | Default Value |
|---|---|
| **int / other integers** | 0 |
| **float / double** | 0.0 |
| **boolean** | false |
| **String / any reference type** | null |

**null means "nothing."** It's the default for any non-primitive (reference) type, since String and other objects belong to classes — you can't give them 0 the way you can a number.

### Local Variables Have NO Default Values

**Instance variables (heap) get defaults; local variables (stack) do not.** `int x; System.out.println(x);` inside a function throws "*local variable x may not have been initialized*." Reason: Java optimizes this way — objects tend to live and get reused for a long time, so giving them safe defaults is worth it; a local variable's scope is short-lived and tied to one function call, so Java forces you to assign it explicitly instead of wasting effort on a default.

## Constructors

Manually assigning every field one line at a time (`s1.name = ...; s1.age = ...;`) is tedious and easy to forget a field for. A **constructor** runs automatically the moment an object is created, letting you initialize it right away.

### Rules of a Constructor

- **Same name as the class.**
- **No return type at all** — not even `void`.
- **Called automatically** during object creation.
- Used to **initialize** a new object.
- Can be **overloaded**, just like regular methods.




No return-type slot at all — unlike a normal method, which always has one (even if it's void).

### Constructor Call Is Hidden in Plain Sight

`new Student()` is actually **two things happening**: `new` allocates heap memory, and `Student()` is the constructor **being called** — using the exact same syntax as calling any other method (name + parentheses).

### The Default Constructor

A constructor is **mandatory** for every class. If you don't write one yourself, Java silently inserts an **empty** constructor with no parameters — the **default constructor**. That's why `new Student()` has always worked even when no constructor appeared in the class — Java supplied one behind the scenes. With nothing assigned, every field keeps its default value (null/0/false).

### Parameterized Constructor

Instead of hard-coding fixed values inside the constructor, accept parameters and assign them dynamically:

> `Student(String n, int a, int rn, String c) { name = n; age = a; rollNumber = rn; college = c; }` — now creating an object requires supplying all four: `new Student("Aditya", 28, 1001, "IIT Guwahati")`.

> **Once you write ANY constructor yourself, Java stops generating the default one.** If your class only has a parameterized constructor, `new Student()` (no arguments) now fails to compile — "*constructor Student is undefined*." To support both styles, write both constructors yourself (this is constructor **overloading**).

## Active Recall — Part 1

**Q1. What is the default value of a String field that was never assigned?**

A. null.

**Q2. Why does a local int variable have no default value, unlike an instance int field?**

A. Java optimizes this way — instance variables tend to be reused for a long time so defaults are worth providing, while local variables are short-lived, so Java forces explicit initialization instead.

**Q3. What are the five rules of a constructor?**

A. Same name as the class; no return type (not even void); called automatically on object creation; used to initialize the object; can be overloaded.

**Q4. Why does new Student() stop working once you add a parameterized constructor, unless you also keep a no-argument one?**

A. Because Java only auto-generates the default constructor when the class defines no constructor at all; once you write any constructor, that auto-generation stops.

## The this Keyword

`this` refers to the **current object** being constructed/operated on — it stores a reference to it, exactly like any reference variable (e.g. `s1`).

### Use 1: Disambiguating Same-Named Variables

If a constructor parameter shares a name with the instance field (common and preferred style), writing `name = name;` is ambiguous — Java just assigns the field to itself (a "*has no effect*" warning), since without a qualifier it reads both sides as the same local parameter.

> `this.name = name;` — `this.name` clearly means the field; the bare `name` means the parameter. This is why same-named parameters are preferred over made-up short names: the code reads cleanly and consistently.

### Use 2: Calling One Constructor From Another (Chaining)

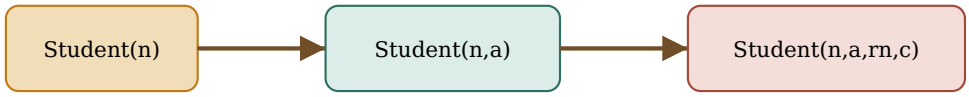
When a class has several overloaded constructors building up to one "biggest" constructor that sets everything,

smaller constructors can **delegate** to a bigger one instead of repeating field-assignment logic:

`Student(String name) { this(name, 0, 0, null); }` — calls the four-argument constructor directly, passing sensible defaults for the rest.

**Hard rule:** a `this(...)` constructor call must be the **very first statement** in the constructor — nothing (not even another statement) can precede it, or it's a compile error.

Chaining can also run the other direction — the **smallest** constructor calls the next bigger one, which calls the next, and so on, up to the biggest. Either direction, calls cascade down (or up) the chain and then **execute in reverse** as each call returns — the same unwind pattern seen with function chaining and recursion.




this(...) chains constructors so default values live in one place (the biggest constructor), avoiding repetition.

## Active Recall — Part 2

**Q1. What does this always refer to?**

A. The current object — a reference to it, same as any reference variable.

**Q2. Why is this.name = name; needed instead of name = name;?**

A. Without the qualifier, Java treats both sides as the same parameter (no effect); this.name specifically targets the instance field.

**Q3. What restriction applies to a this(...) constructor call?**

A. It must be the first statement in the constructor.

**Q4. Can a constructor be called manually like a normal method, e.g. mid-program?**

A. No — a constructor can only be invoked when an object is first created with new, or via this(...) from within another constructor.

**Q5. Can object creation with new ever fail at run time due to memory?**

A. Yes — new performs dynamic (run-time) memory allocation in the heap; if the heap doesn't have enough free space, a run-time exception occurs.

## How Big Is an Object in Memory?

A reference variable (like `s1`) sits in the stack and typically takes **4 bytes** (sometimes 8, depending on the JVM's internal compression — most commonly 4). But the **object itself** in the heap is made of three parts:

| Object Header | Exact Data | Padding |
|---|---|---|
| Metadata — mark word + class pointer | The actual field values | Rounds total size up to a multiple of 8 bytes |

### Object Header (Metadata)

- **Mark word (8 bytes)** — stores info like locking mechanism, synchronization state, garbage collection info (all covered later, once multithreading/GC are studied).
- **Class pointer (4 bytes, typically)** — a reference back to the object's class, conceptually similar to any other

reference variable.

Total header: **8 + 4 = 12 bytes** (using the common 4-byte reference size).

### Exact Data

The actual fields' sizes, added up. Example — a Student with `String name, int age, int rollNumber, String college`: name (4, reference) + age (4, int) + rollNumber (4, int) + college (4, reference) = **16 bytes**.

### Padding

Modern CPUs fetch memory in **8-byte chunks**, so Java tries to make every object's total size a **multiple of 8**. Add header + exact data; if the result isn't already a multiple of 8, pad up to the next one.




12 + 16 = 28 → nearest multiple of 8 above 28 is 32 → pad with 4 bytes.

> **Worked example 2:** a class with a single `byte age` field: header 12 + exact data 1 (byte) = 13; nearest multiple of 8 above 13 is 16 → pad 3 bytes → total **16 bytes**... (if only one field) or as calculated per the actual class — the method always: sum header + data, find the gap to the next multiple of 8, add that as padding.

## Active Recall — Part 3

**Q1. What three components make up an object's total size in the heap?**

A. Object header (mark word + class pointer), exact field data, and padding.

**Q2. Why does Java pad object size to a multiple of 8 bytes?**

A. Because modern CPUs fetch memory in 8-byte chunks, so aligning object size to that boundary is more efficient to process.

**Q3. What two things make up the mark word, roughly?**

A. Metadata like locking/synchronization info and garbage collection info (studied in depth later).

## Call by Value vs Call by Reference

Passing `int x, y` to a function and modifying them inside does **not** change the originals — because Java creates **brand-new local variables** (the parameters) and **copies** the argument values into them. Changes to the copies vanish when the function returns.

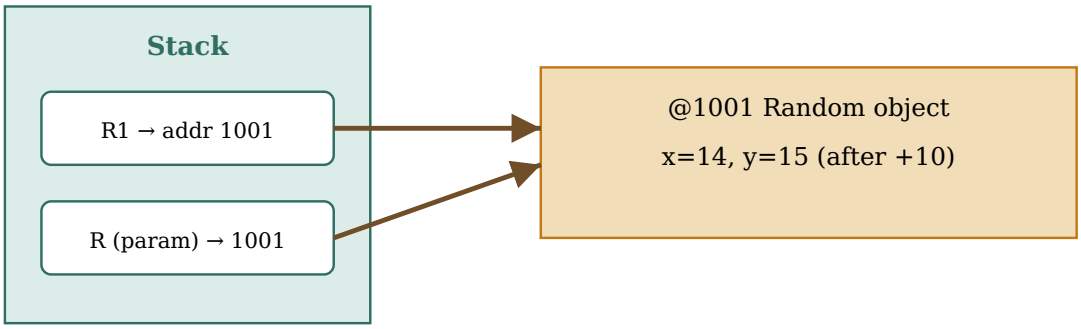
> **This is Call by Value** — only the value is copied in, never a link back to the original variable.

### Objects Look Different — But Aren't

Passing an **object reference** (e.g. a `Random` object with fields x, y) to a function and modifying `r.x` / `r.y` inside **does** change the original object. This looks like call-by-reference, but it's still call-by-value underneath:

- The parameter (`r`) is a **new, separate reference variable** — its value is **copied** from the argument, exactly as with primitives.
- But what gets copied is an **address** (the heap location of the object) — so the new reference variable ends up pointing to the **exact same object** as the original.

- Changing a field through either reference changes the one shared object in the heap — both references read/write the same data.



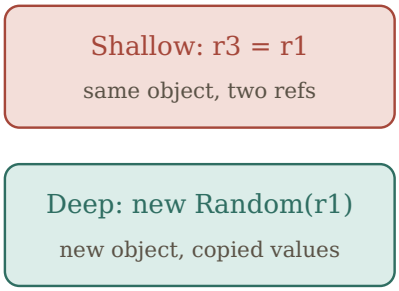

R1 and R are two separate reference variables, but both store the same address — so changes through either reach the same object.

**Java has NO call-by-reference — period.** It is call-by-value everywhere, for primitives and objects alike. Objects merely give you a call-by-reference *effect*, because the value being copied is an address, not the data itself. Claiming "Java is call-by-value for primitives and call-by-reference for objects" is a common interview mistake — technically wrong.

## Shallow Copy vs Deep Copy

Suppose you want a second reference/object with the same field values as an existing one:

- **Shallow copy** — `Random r3 = r1;` — no `new` keyword, so no new object is created. `r3` simply copies `r1`'s stored address, so both now point to the **exact same object**. Changing a field through either reference affects both.
- **Deep copy** — e.g. a constructor `Random(Random r) { this.x = r.x; this.y = r.y; }` called as `new Random(r1)` — creates a genuinely **new, independent object** in the heap and copies the field *values* across. The two objects are now completely separate; changing one never affects the other.




Shallow copy shares state; deep copy duplicates it independently.

**Choosing between them:** use shallow copy when two references should legitimately refer to the same underlying object (shared state intended); use deep copy when you need a fully independent duplicate that can change without affecting the original.

## Active Recall — Part 4

**Q1. Why does modifying a primitive parameter inside a function not affect the caller's variable?**

A. The parameter is a new local variable, and only the value is copied in (call by value) — changes to the copy don't touch the original.

**Q2. Why does modifying an object's field inside a function appear to change the original object?**

A. The parameter is a separate reference variable, but the value copied into it is the object's address — so it points to the same heap object as the original, and field changes are visible through both.

**Q3. Is Java ever truly call by reference?**

A. No — Java is call by value everywhere; objects just produce a call-by-reference-like effect because addresses (not raw data) get copied.

**Q4. What's the key difference between shallow copy (r3 = r1) and deep copy (new Random(r1))?**

A. Shallow copy makes two references point to the same object (shared changes); deep copy creates a genuinely separate object with copied values (independent changes).

---